

Java - Introducing Classes

Lecture 8

Instructor:

Shovon Mandal (SM), Lecturer

Department of Computer Science and Engineering (CSE),
Northern University & Business Technology Khulna (NUBTK)

Classes

8.1

What is class in Java?

- The class is at the core of Java.
- It is the logical construct upon which the entire Java language is built because it defines the shape and nature of an object.
- As such, the class forms the basis for object-oriented programming in Java. Any concept you wish to implement in a Java program must be encapsulated within a class.
- When you create a class, you are creating a new data type.
- Classes usually consist of two things:
 - Instance variables and
 - Methods

General Form of a Class

- When you define a class, you declare its exact form and nature.
- A class is declared by use of the class keyword.
- A simplified general form of a class definition is shown here:

```
class classname {  
    type instance-variable1;  
    type instance-variable2;  
    // ...  
    type instance-variableN;  
  
    type methodname1(parameter-list) {  
        // body of method  
    }  
    type methodname2(parameter-list) {  
        // body of method  
    }  
    // ...  
    type methodnameN(parameter-list) {  
        // body of method  
    }  
}
```

A Simple Class

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

- Here is a class called Box
- that defines three instance variables: width, height, and depth. Currently, Box does
- not contain any methods (but some will be added soon).

```
Box mybox = new Box(); // create a Box object called mybox
```

- After this statement executes, mybox will refer to an instance of Box.
- Every Box object will contain its own copies of the instance variables width, height, and depth. To access these variables, you will use the dot (.) operator. The dot operator links the name of the object with the name of an instance variable. For example, to assign the width variable of mybox the value 100, you would use the following statement:

```
mybox.width = 100;
```
- This statement tells the compiler to assign the copy of width that is contained within the mybox object the value of 100.

A Simple Class cont.

```
/* A program that uses the Box class.

   Call this file BoxDemo.java
*/
class Box {
    double width;
    double height;
    double depth;
}

// This class declares an object of type Box.
class BoxDemo {
    public static void main(String args[]) {
        Box mybox = new Box();
        double vol;

        // assign values to mybox's instance variables
        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;

        // compute volume of box
        vol = mybox.width * mybox.height * mybox.depth;

        System.out.println("Volume is " + vol);
    }
}
```

- You should call the file that contains this program BoxDemo.java, because the main() method is in the class called BoxDemo, not the class called Box.
- When you compile this program, you will find that two .class files have been created, one for Box and one for BoxDemo.
- The Java compiler automatically puts each class into its own .class file. It is not necessary for both the Box and the BoxDemo class to actually be in the same source file. You could put each class in its own file, called Box.java and BoxDemo.java, respectively. To run this program, you must execute BoxDemo.class. When you do, you will see the following output:

```
Volume is 3000.0
```

A Simple Class cont.

- Each object has its own copies of the instance variables.
- If you have two Box objects, each has its own copy of depth, width, and height.

The output produced by this program is shown here:

```
Volume is 3000.0
Volume is 162.0
```

```
// This program declares two Box objects.
```

```
class Box {
    double width;
    double height;
    double depth;
}

class BoxDemo2 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;

        /* assign different values to mybox2's
           instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;

        // compute volume of first box
        vol = mybox1.width * mybox1.height * mybox1.depth;
        System.out.println("Volume is " + vol);

        // compute volume of second box
        vol = mybox2.width * mybox2.height * mybox2.depth;
        System.out.println("Volume is " + vol);
    }
}
```

Declaring Objects

Steps to Create an Object:

1. Declare a Variable of the Class Type:

ClassName obj;

- This variable is a reference, not the object itself.

2. Acquire an Object Using `new` :

- The `new` operator dynamically allocates memory at runtime.
 - Returns a reference (memory address) to the allocated object.
- Example:

obj = new ClassName();

How `new` Works:

- Dynamic Allocation: Memory is reserved during program execution.
- Allocates memory during runtime.
- Reference Storage: The reference (address) of the object is stored in the variable.
- Key Point: All class objects in Java must be dynamically allocated using `new`.

```
Box mybox = new Box();
```

```
Box mybox; // declare reference to object  
mybox = new Box(); // allocate a Box object
```


Declaring Objects Cont.

```
Box mybox = new Box();
```

Statement

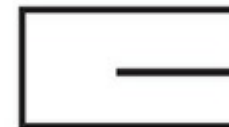
Effect

Box mybox;

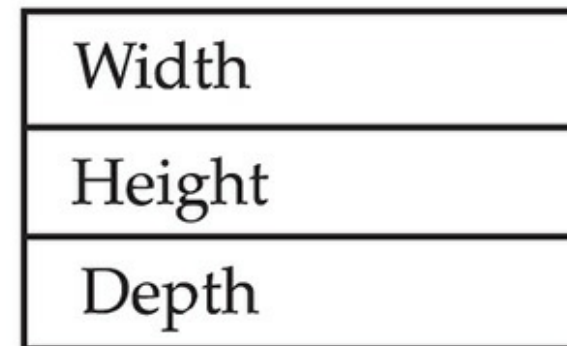


mybox

mybox = new Box();



mybox

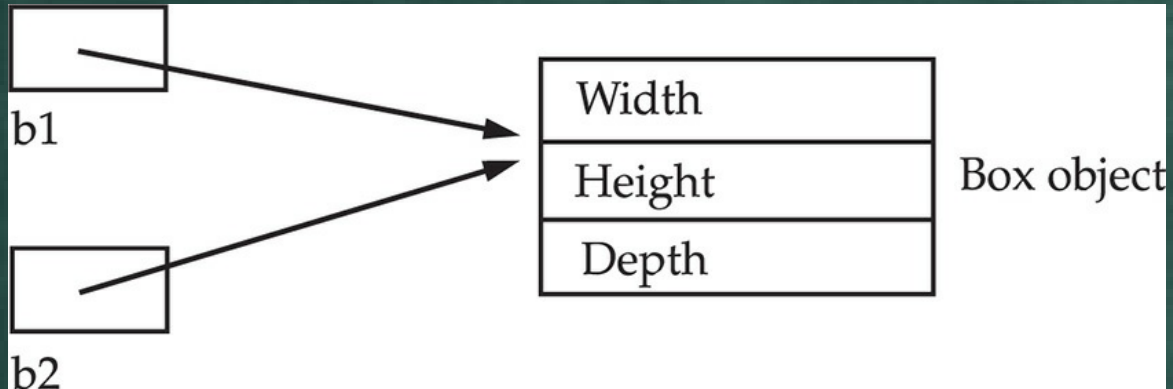


Box object

Assigning Object Reference Variables

```
Box b1 = new Box();  
Box b2 = b1;
```

```
Box b1 = new Box();  
Box b2 = b1;  
// ...  
b1 = null;
```



b1 has been set to null, but b2 still points to the original object.

b1 and b2 both refer to the same object

Important: When you assign one object reference variable to another object reference variable, you are not creating a copy of the object, you are only making a copy of the reference.

Methods

8.2

What is a Method?

- A block of code designed to perform a specific task.
- Encapsulation of logic inside a class.
- Key Features of methods:
 - **Reusable:** Can be called multiple times from different parts of a program.
 - **Parameters:** Accept input data (optional).
 - **Return Type:** Can return a value or void if no value is returned.
- Types of Methods
 - **Instance Methods:** Require an object to be called.
 - **Static Methods:** Belong to the class and can be called without creating an object (e.g., `Math.sqrt()`).
- Benefits:
 - **Code Reusability:** Write once, use multiple times.
 - **Modularity:** Break code into smaller, manageable units.
 - **Readability:** Improve code clarity.

Introducing Methods

- General Form of a method:

```
type name(parameter-list) {  
    // body of method  
}
```

- Type specifies the type of data returned by the method.
- If the method does not return a value, its return type must be void.
- The parameter-list is a sequence of type and identifier pairs separated by commas.

- Methods that have a return type other than void return a value to the calling routine using the following form of the return statement:

return value;

- Here, value is the value returned.

Adding a Method to a Class

```
// This program includes a method inside the box class.
```

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    // display volume of a box  
    void volume() {  
        System.out.print("Volume is ");  
        System.out.println(width * height * depth);  
    }  
}
```

```
Volume is 3000.0  
Volume is 162.0
```

```
class BoxDemo3 {  
    public static void main(String args[]) {  
        Box mybox1 = new Box();  
        Box mybox2 = new Box();  
  
        // assign values to mybox1's instance variables  
        mybox1.width = 10;  
        mybox1.height = 20;  
        mybox1.depth = 15;  
  
        /* assign different values to mybox2's  
           instance variables */  
        mybox2.width = 3;  
        mybox2.height = 6;  
        mybox2.depth = 9;  
  
        // display volume of first box  
        mybox1.volume();  
  
        // display volume of second box  
        mybox2.volume();  
    }  
}
```

Returning a Value

```
// Now, volume() returns the volume of a box.

class Box {
    double width;
    double height;
    double depth;

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}

class BoxDemo4 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;
```

```
        /* assign different values to mybox2's
           instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}
```

- **One More way:**

```
System.out.println("Volume is" + mybox1.volume());
```

Method That Takes Parameters

- Parameters allow a method to be generalized.
- That is, a parameterized method can operate on a variety of data and/or be used in a number of slightly different situations.

```
int square()  
{  
    return 10 * 10;  
}
```

```
int square(int i)  
{  
    return i * i;  
}
```

```
int x, y;  
x = square(5); // x equals 25  
x = square(9); // x equals 81  
y = 2;  
x = square(y); // x equals 4
```

Remember the box Class:

```
mybox1.width = 10;  
mybox1.height = 20;  
mybox1.depth = 15;
```

Method That Takes Parameters (On Box Class)

```
// This program uses a parameterized method.
```

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    // compute and return volume  
    double volume() {  
        return width * height * depth;  
    }  
  
    // sets dimensions of box  
    void setDim(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
}
```

```
class BoxDemo5 {  
    public static void main(String args[]) {  
        Box mybox1 = new Box();  
        Box mybox2 = new Box();  
        double vol;  
  
        // initialize each box  
        mybox1.setDim(10, 20, 15);  
        mybox2.setDim(3, 6, 9);  
  
        // get volume of first box  
        vol = mybox1.volume();  
        System.out.println("Volume is " + vol);  
  
        // get volume of second box  
        vol = mybox2.volume();  
        System.out.println("Volume is " + vol);  
    }  
}
```

Constructors

8.3

What is a Constructor?

- A special method used to initialize objects when they are created.
- Key Features:
 - Same name as the class.
 - No return type (not even void).
 - Automatically called when the object is created.
- Purpose of Constructors:
 - Simplifies initialization by performing setup at creation time.
 - Ensures objects are fully initialized and usable immediately.

```
class Box {  
    double width, height, depth;  
  
    // Constructor  
    Box() {  
        width = 10;  
        height = 10;  
        depth = 10;  
    }  
}
```

Benefits:

- Eliminates repetitive initialization code.
- Replaces manual methods like setDim().

Constructor - Box Class

```
/* Here, Box uses a constructor to initialize the
   dimensions of a box.
*/
class Box {
    double width;
    double height;
    double depth;

    // This is the constructor for Box.
    Box() {
        System.out.println("Constructing Box");
        width = 10;
        height = 10;
        depth = 10;
    }

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}
```

```
class BoxDemo6 {
    public static void main(String args[]) {
        // declare, allocate, and initialize Box objects
        Box mybox1 = new Box();
        Box mybox2 = new Box();

        double vol;

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}
```

```
Constructing Box
Constructing Box
Volume is 1000.0
Volume is 1000.0
```

Parameterized Constructors

- What is a Parameterized Constructor?
 - A constructor that accepts parameters to initialize an object with specific values.

```
class ClassName {  
    // Fields  
    int field;  
  
    // Parameterized Constructor  
    ClassName(int value) {  
        field = value;  
    }  
}
```

Parameterized Constructors - Box Class

```
/* Here, Box uses a parameterized constructor to
   initialize the dimensions of a box.
*/
```

```
class Box {
    double width;
    double height;
    double depth;

    // This is the constructor for Box.
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }
}
```

```
    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}
```

```
class BoxDemo7 {
    public static void main(String args[]) {
        // declare, allocate, and initialize Box objects
        Box mybox1 = new Box(10, 20, 15);
        Box mybox2 = new Box(3, 6, 9);

        double vol;

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}
```

```
Volume is 3000.0
Volume is 162.0
```

The this Keyword

- What is **this** Keyword?
 - **this** is a reference variable in Java that refers to the current object.
 - Used to differentiate between instance variables and parameters with the same name.
- When to Use this?
 - To resolve variable shadowing (when instance variables and parameters have the same name).
 - To call one constructor from another within the same class.
 - To pass the current instance as an argument to another method.

```
class Box {  
    double width;  
    double height;  
    double depth;  
    // Constructor using 'this' to refer to instance variables  
    Box(double width, double height, double depth) {  
        this.width = width;  
        this.height = height;  
        this.depth = depth;  
    }  
    // Method to calculate volume  
    double volume() {  
        return width * height * depth;  
    }  
}  
  
class BoxDemo {  
    public static void main(String[] args) {  
        Box myBox = new Box(10, 20, 15);  
        System.out.println("Volume is " + myBox.volume());  
    }  
}
```

Java Scanner Class

- What is the Scanner Class?
 - Part of java.util package.
 - Used to read input from various sources (keyboard, files, strings).
 - Simplifies reading primitive types and strings.
- Features:
- Can read multiple data types.
- By default, whitespace is the delimiter, but it can be customized.
- Throws exceptions (e.g., InputMismatchException) if input doesn't match the expected type.

Method	Description
<code>nextBoolean()</code>	Reads a <code>boolean</code> value from the user
<code>nextByte()</code>	Reads a <code>byte</code> value from the user
<code>nextDouble()</code>	Reads a <code>double</code> value from the user
<code>nextFloat()</code>	Reads a <code>float</code> value from the user
<code>nextInt()</code>	Reads a <code>int</code> value from the user
<code>nextLine()</code>	Reads a <code>String</code> value from the user
<code>nextLong()</code>	Reads a <code>long</code> value from the user
<code>nextShort()</code>	Reads a <code>short</code> value from the user

Example of Scanner

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter an integer: ");
        int num = sc.nextInt();

        System.out.print("Enter a string: ");
        String str = sc.next();

        System.out.println("Integer: " + num);
        System.out.println("String: " + str);

    }
}
```

- Commonly Used Methods

- `nextInt()`: Reads an integer.
- `nextDouble()`: Reads a double.
- `nextLine()`: Reads a line of text.
- `next()`: Reads the next token (word).
- `nextBoolean()`: Reads a boolean value.

Java-Power of classes [Stack]

- Encapsulation: Combines data and methods in one unit, providing a consistent and controlled interface.
- A class is like a "data engine" that hides internal details.
- Users interact with the class through its methods without needing to know the implementation details.
- Internal workings can be changed without impacting external code.
- What is a Stack?
- A First-In, Last-Out (FILO) data structure.
- Analogy: Like a stack of plates—the first plate added is the last to be removed.
- Operations:
 - Push: Adds an item to the top.
 - Pop: Removes the item from the top.

Java-Power of classes [Stack] Cont.

```
// This class defines an integer stack that can hold 10 values
class Stack {
    int stck[] = new int[10];
    int tos;

    // Initialize top-of-stack
    Stack() {
        tos = -1;
    }

    // Push an item onto the stack
    void push(int item) {
        if(tos==9)
            System.out.println("Stack is full.");
        else
            stck[++tos] = item;
    }

    // Pop an item from the stack
    int pop() {
        if(tos < 0) {
            System.out.println("Stack underflow.");
            return 0;
        }
        else
            return stck[tos--];
    }
}
```

```
class TestStack {
    public static void main(String args[]) {
        Stack mystack1 = new Stack();
        Stack mystack2 = new Stack();

        // push some numbers onto the stack
        for(int i=0; i<10; i++) mystack1.push(i);
        for(int i=10; i<20; i++) mystack2.push(i);

        // pop those numbers off the stack
        System.out.println("Stack in mystack1:");
        for(int i=0; i<10; i++)
            System.out.println(mystack1.pop());

        System.out.println("Stack in mystack2:");
        for(int i=0; i<10; i++)
            System.out.println(mystack2.pop());
    }
}
```

```
Stack in mystack1:
9
8
7
6
5
4
3
2
1
0
Stack in mystack2:
19
18
17
16
15
14
13
12
11
10
```

Stack Class Structure

- Data Items:
 - `int[] stck` (array to hold stack elements).
 - `int tos` (tracks the top of the stack).
- Constructor: Initializes `tos` to `-1` (empty stack).
- Methods:
 - `push(int item)`: Adds an item to the stack.
 - `pop()`: Removes an item from the stack.
- Benefits of Stack Encapsulation
 - Users interact only through `push(int item)` and `pop()`.
 - The stack could be stored in an array, linked list, etc.
 - The internal structure is irrelevant to users.

Note: Encapsulation protects data and ensures safe access.

Thank You

04-Dec-25